Assignment 10

Dockerize & Deploy an Express.js App using Docker Compose with Nginx

Submitted by: **Mahmudur Rahman**

Date: 24 June 2026

Source repository:
https://github.com/roy35-909/Module-3-deployment

DockerHub image:
https://hub.docker.com/r/mrkolince/mahmud_assignment10

This document is both the assignment submission and a step-by-step guide — follow the steps in order to reproduce the entire setup from scratch.

Objective

Containerize a simple Express.js server, publish its image to DockerHub, and run it alongside an Nginx container using Docker Compose. The Express app must start **after** Nginx and be reachable in the browser on **port 8080**.

What you need before starting (prerequisites)

- **Docker Desktop** installed and running on Windows (includes Docker Engine + Docker Compose v2).
- A **DockerHub account**. In this guide the username is **mrkolince** — replace it with your own everywhere it appears.
- **Git** installed (to clone the source repository).
- Any terminal — this guide uses **Windows PowerShell**.

How the two services fit together

Docker Compose builds the Express app from our **Dockerfile** and pulls a stock **nginx:latest** image. The app container maps host port **8080** to container port **5000** (where Express listens). The **depends_on** setting makes Compose start Nginx first, then the Express app.

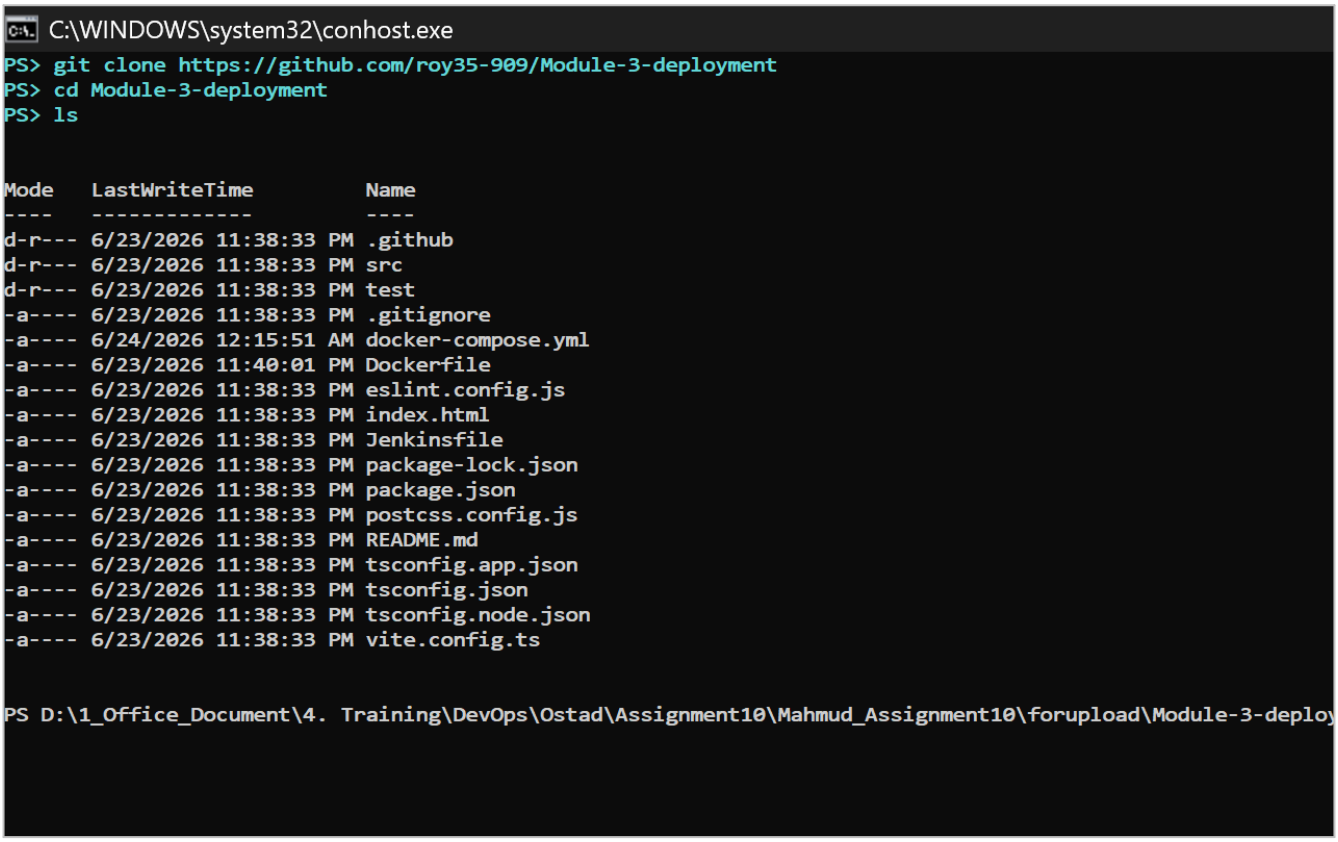
Step 1 — Clone the repository and inspect the project

Clone the provided Express.js project and move into the folder. Listing the contents confirms the files we will work with — most importantly we will add a **Dockerfile** and a **docker-compose.yml** here.

```
git clone https://github.com/roy35-909/Module-3-deployment
```

```
cd Module-3-deployment
```

```
ls
```

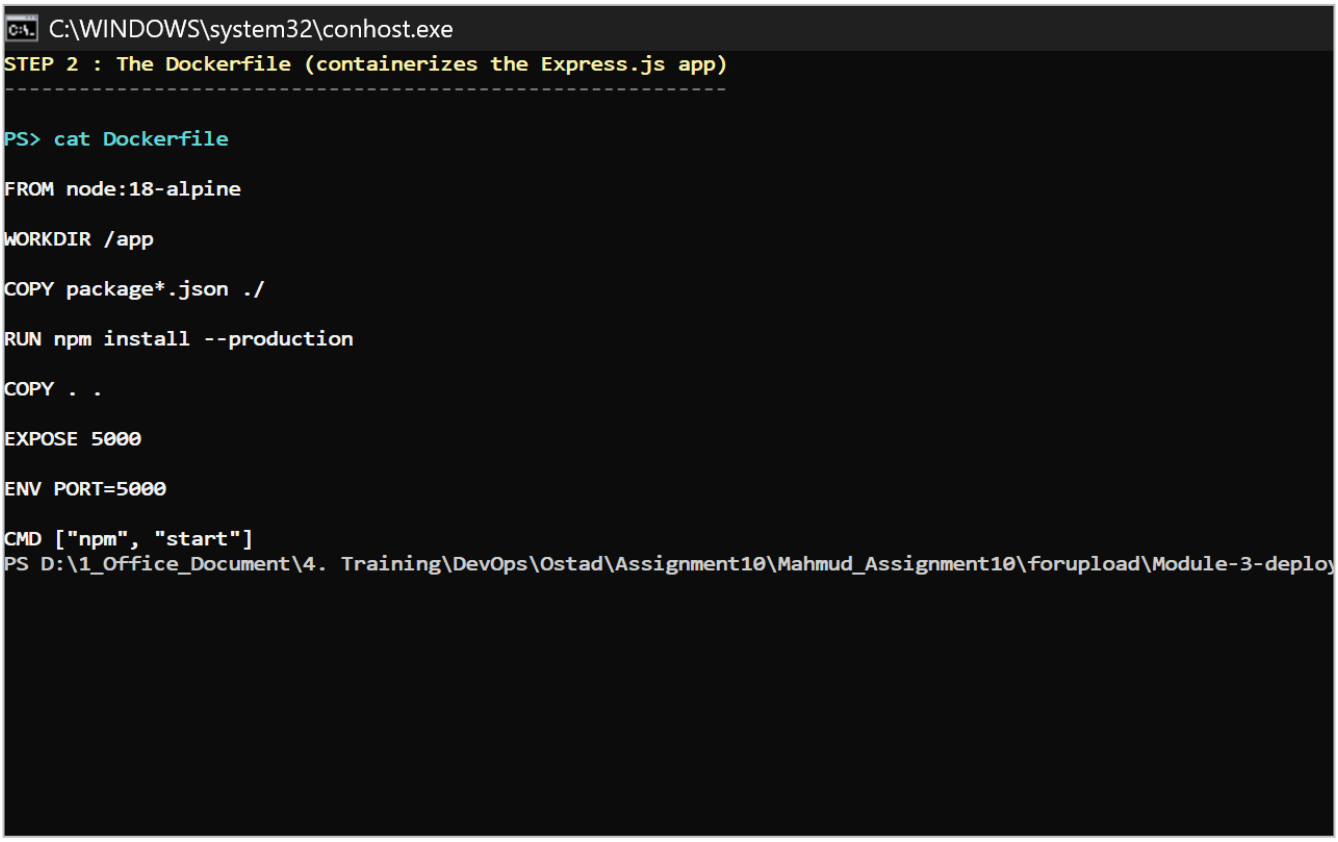


The cloned project files.

Step 2 — The Dockerfile (containerize the Express app)

Create a file named **Dockerfile** in the project root with the content shown below. It starts from a lightweight Node 18 image, installs only production dependencies, copies the source, exposes port 5000 and starts the app with `npm start`.

```
cat Dockerfile
```

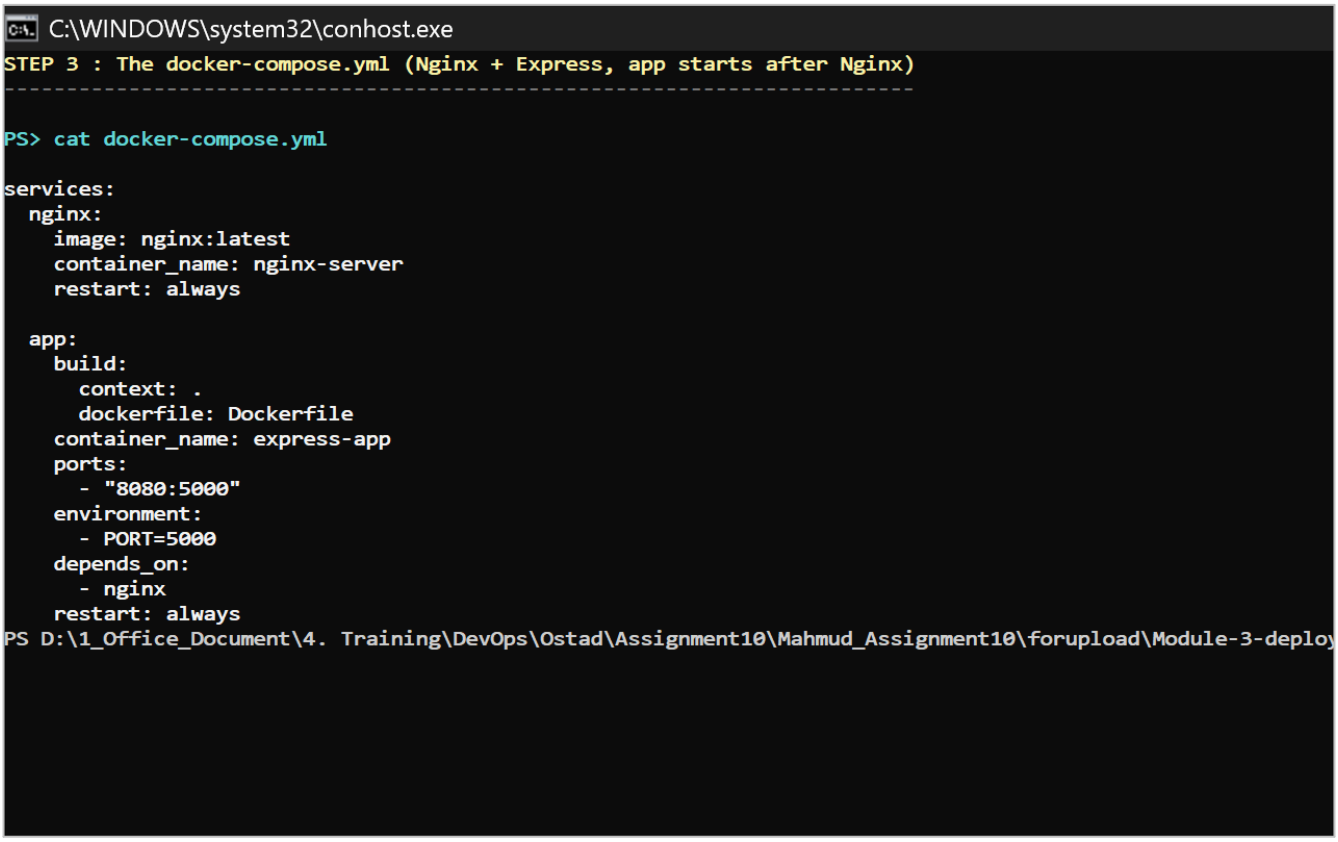


The Dockerfile used to build the Express.js image.

Step 3 — The docker-compose.yml

Create **docker-compose.yml** in the project root. It defines two services: **nginx** (a plain nginx:latest image) and **app** (built from our Dockerfile). The app maps `8080:5000` and uses `depends_on: nginx` so the Express app starts after Nginx.

```
cat docker-compose.yml
```



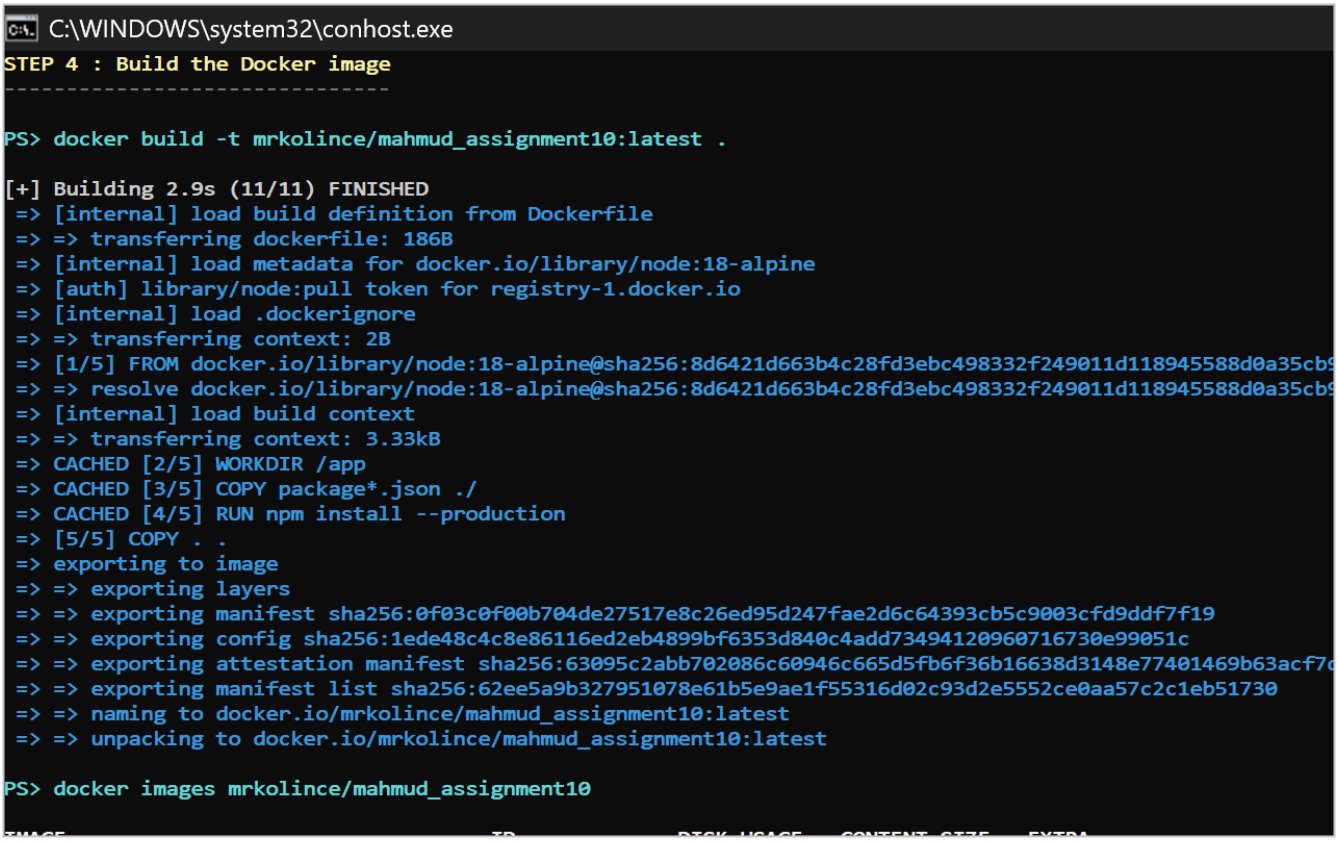
The docker-compose.yml defining Nginx + Express.

Step 4 — Build the Docker image

Build the image and tag it with your DockerHub namespace so it is ready to push. After the build, `docker images` confirms the image exists locally (~192 MB).

```
docker build -t mrkolince/mahmud_assignment10:latest .
```

```
docker images mrkolince/mahmud_assignment10
```



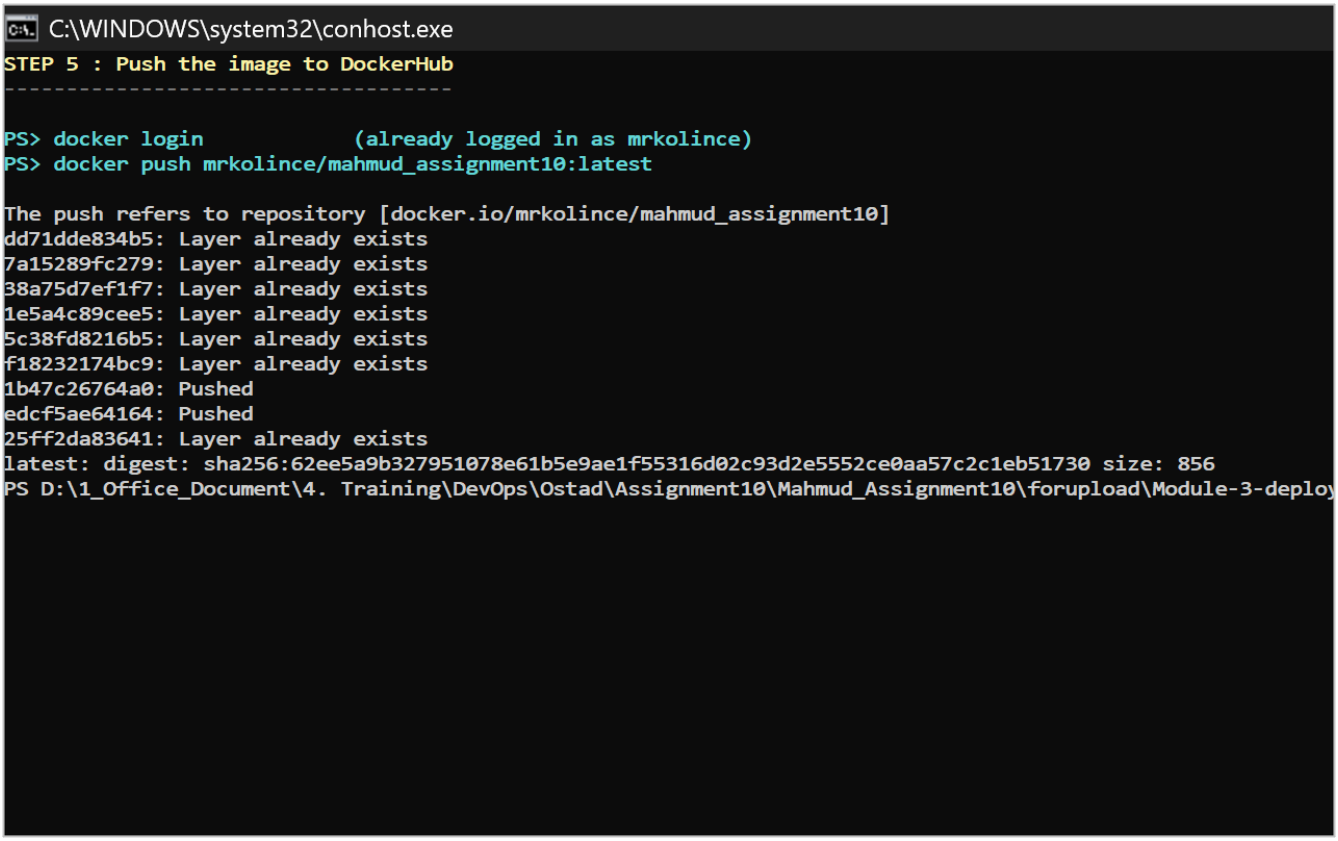
A successful build; the image is listed at the bottom.

Step 5 — Push the image to DockerHub

Log in once with `docker login` (enter your DockerHub username and password when prompted), then push the tagged image. The final `digest: sha256:...` line confirms the upload succeeded.

```
docker login
```

```
docker push mrkolince/mahmud_assignment10:latest
```

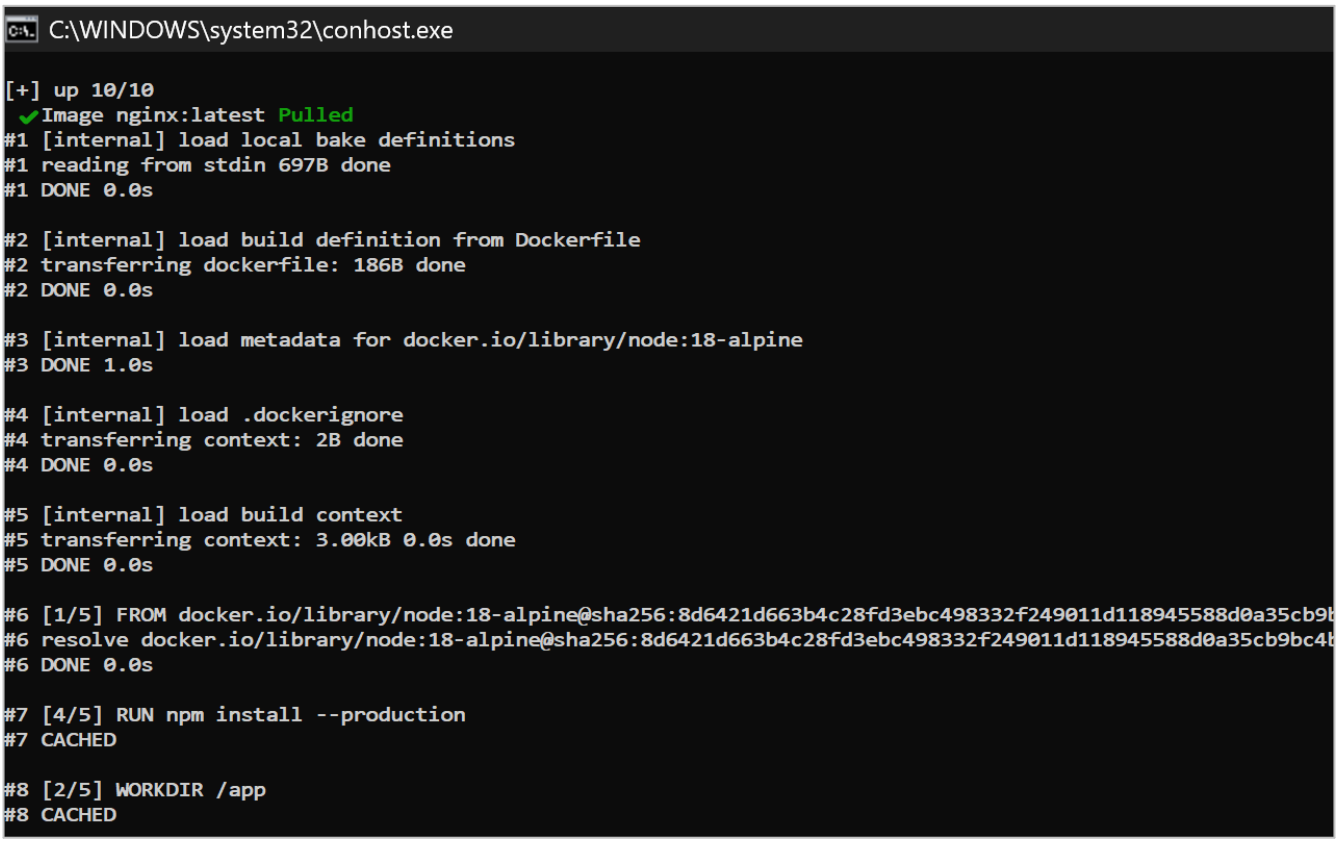


The image layers are pushed and the digest is returned.

Step 6 — Start everything with Docker Compose

From the project folder, bring up the whole stack in the background. Compose pulls nginx:latest, builds the app image, creates a network, and starts both containers — Nginx first, then the Express app.

```
docker compose up -d
```



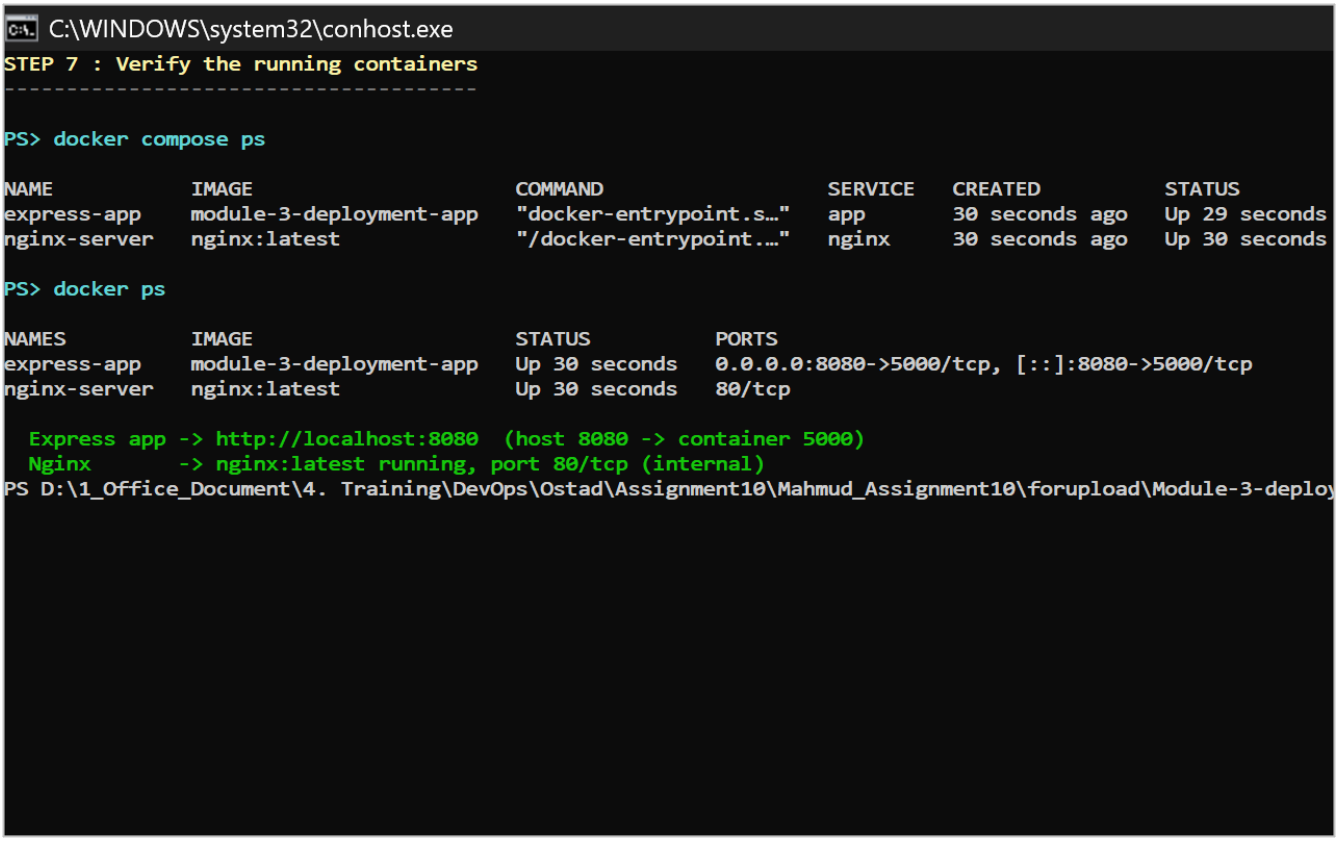
Nginx pulled, app built, both containers started.

Step 7 — Verify the running containers

Confirm both containers are **Up**. Note the app's port mapping `0.0.0.0:8080->5000/tcp` and that nginx-server exposes 80/tcp internally.

```
docker compose ps
```

```
docker ps
```

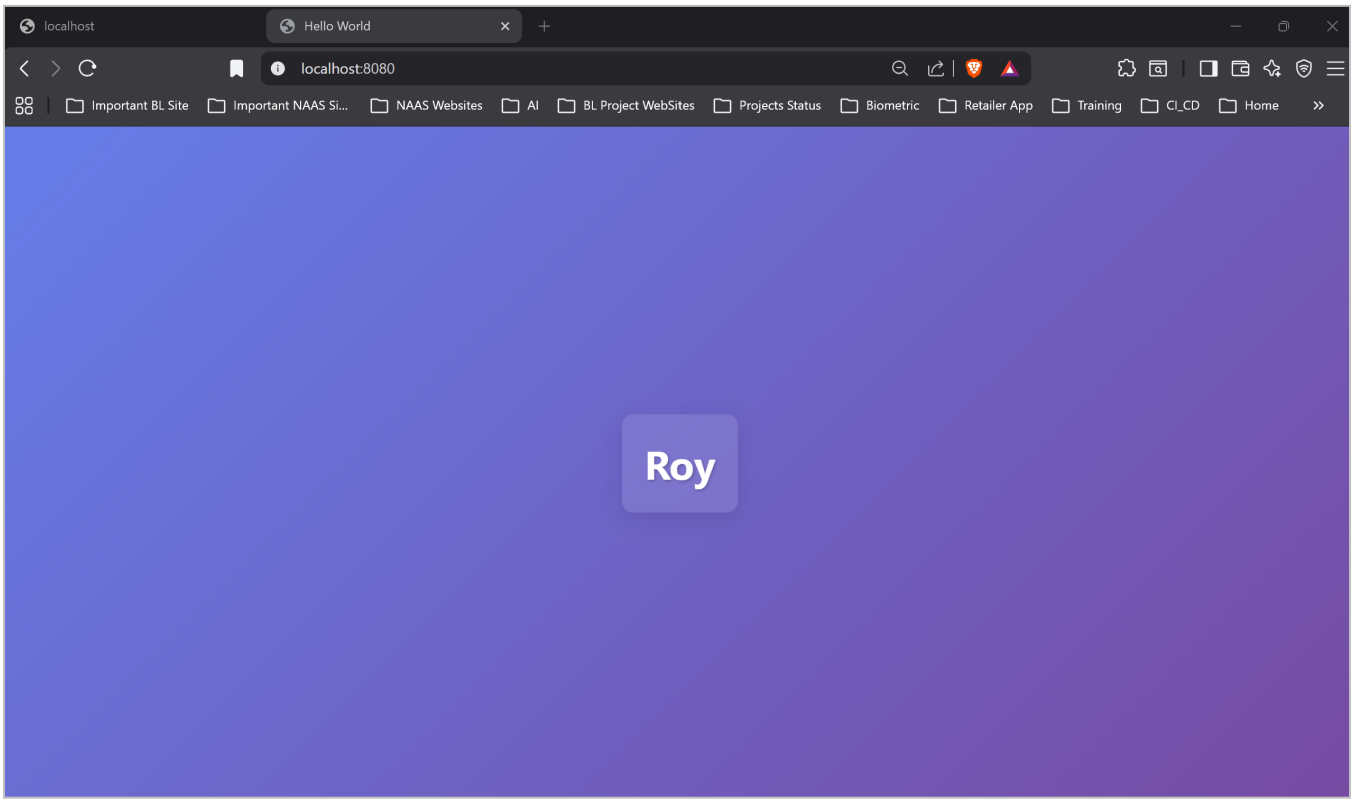


Both containers running; app exposed on 8080.

Step 8 — Verify it works in the browser

Open a browser and visit **http://localhost:8080**. The Express app responds — proving the containerized application is reachable on the required port 8080.

```
start http://localhost:8080
```

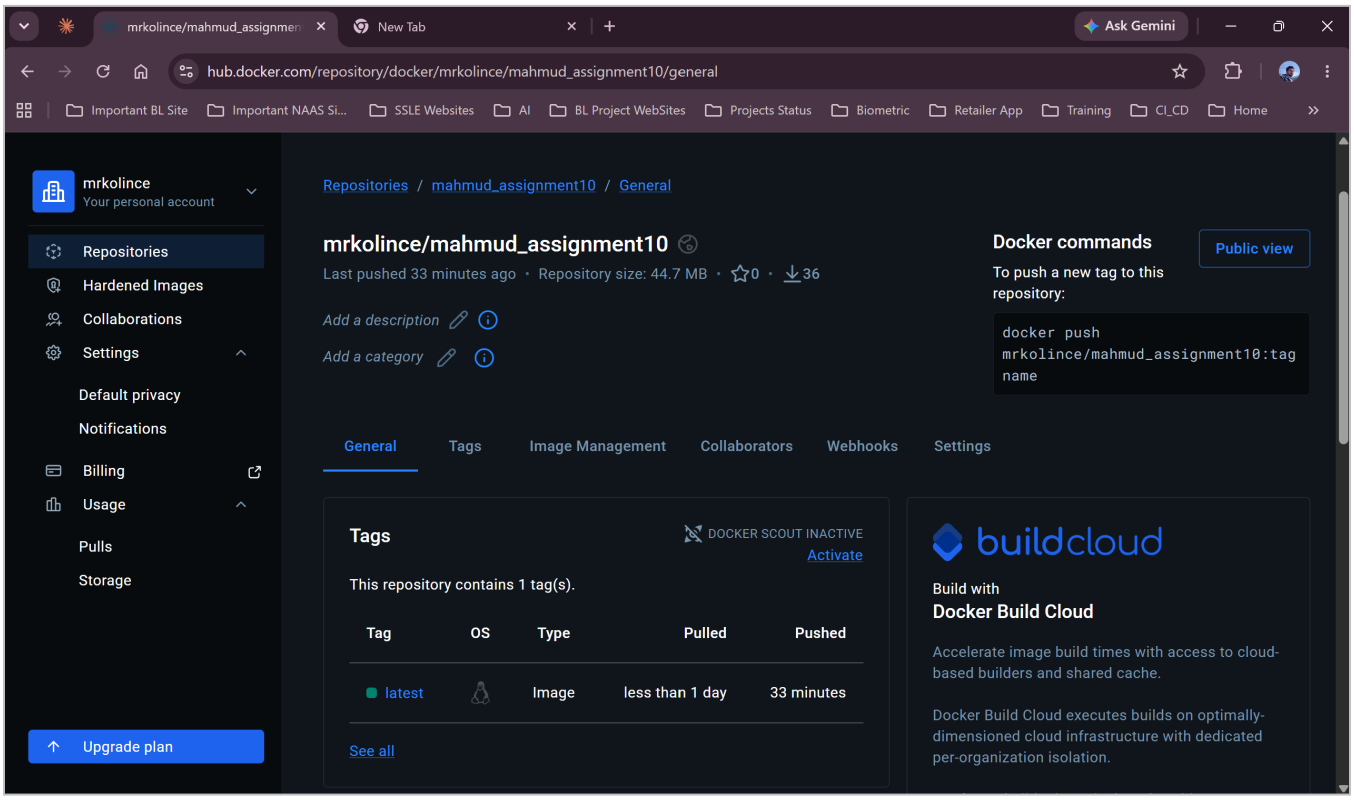


The app served from the container at localhost:8080.

Step 9 — Confirm the image on DockerHub

Finally, open your DockerHub repository to confirm the image was published. The **latest** tag is present and anyone can now pull it with the command shown on the page.

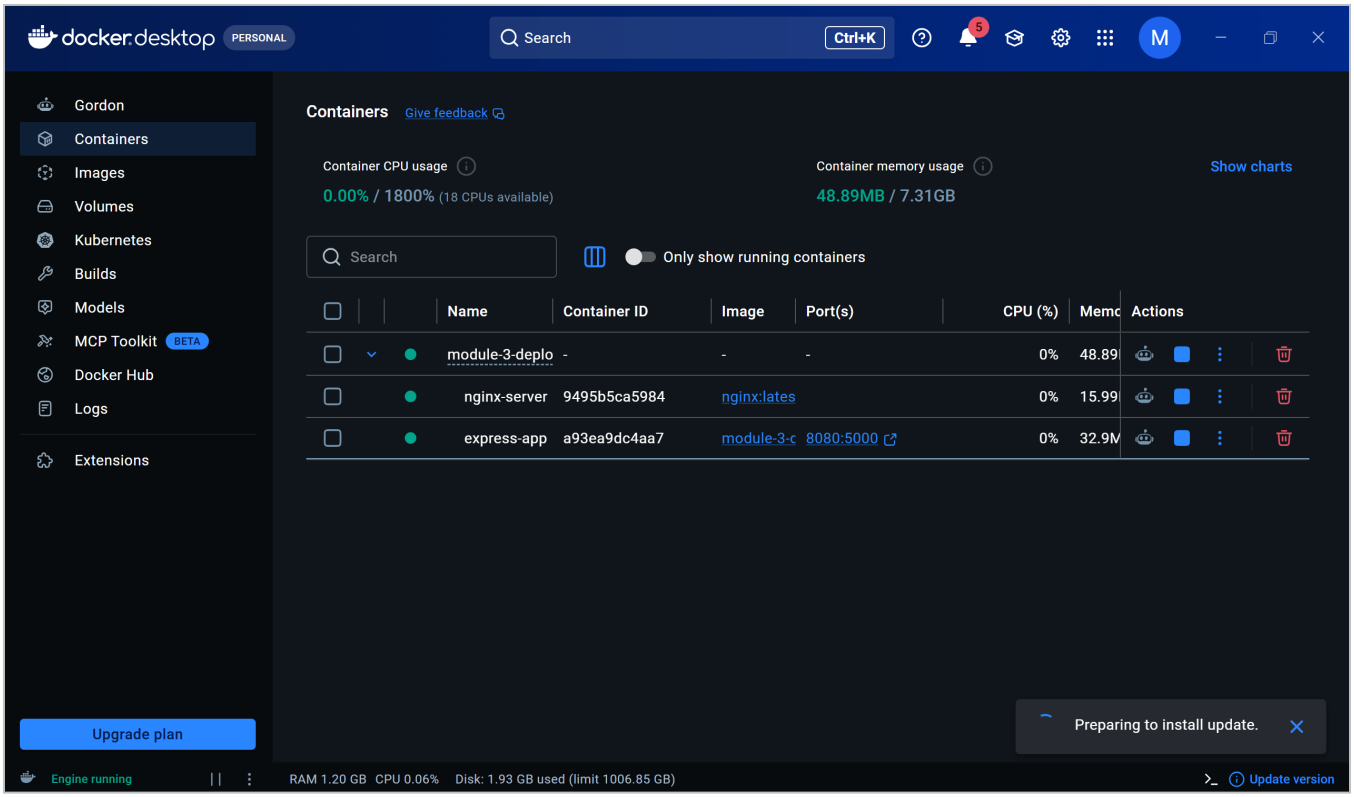
```
docker pull mrkolince/mahmud_assignment10:latest
```



The published image on DockerHub (latest tag).

Step 10 — Docker Desktop view (optional)

Docker Desktop → **Containers** shows the running `module-3-deployment` Compose stack with a green ‘running’ indicator — a visual confirmation of the same state as the terminal in Step 7.



The Compose stack running, seen in Docker Desktop.

Submission summary

Item	Link / Value
DockerHub image	`https://hub.docker.com/r/mrkolince/mahmud_assignment10`
Pull command	`docker pull mrkolince/mahmud_assignment10:latest`
Git repository	`https://github.com/roy35-909/Module-3-deployment`
Files added to repo	`Dockerfile, docker-compose.yml`
App URL (local)	`http://localhost:8080`
Exposed port	`8080 (host) -> 5000 (container)`

All five assignment objectives are satisfied: the Express app is dockerized and pushed to DockerHub; docker-compose.yml runs an Nginx image and builds/runs the Express app with `depends_on`; the app is exposed on port 8080; everything is run via Docker Compose; and it is verified working in the browser.